

**PROPOSAL PROJECT
PRAKTIKUM RPLBO**



Judul Aplikasi	UangKu
Mata Kuliah	Praktikum RPLBO
Dosen Pengampu	Prof. Antonius Rachmat Chrismanto
Disusun oleh	Ketua: 71241164 Waraney Maikel Nathaniel Mambu Anggota: 71241097 Delvin Laurens 71241126 Valentino Kevin Yulianto 71241163 Jeremy Zadrimman Kause

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

1. Pendahuluan

1.1 Latar Belakang

Banyak mahasiswa dan pekerja muda menghadapi kesulitan dalam mengelola keuangan harian mereka. Seringkali, pengeluaran tidak terkendali karena kurangnya pencatatan yang sistematis, sehingga sulit untuk mengetahui ke mana uang habis di akhir bulan. Pencatatan manual di buku atau notes HP seringkali tidak rapi dan tidak memberikan analisis visual.

Kelompok kami memilih tema ini untuk memberikan solusi digital yang memudahkan pengguna mencatat arus kas (masuk dan keluar), menetapkan anggaran bulanan, serta melihat laporan keuangan secara otomatis. Harapannya, proyek ini dapat membantu pengguna menjadi lebih disiplin secara finansial dan menerapkan prinsip pemrograman berorientasi objek dalam membangun aplikasi yang fungsional.

1.3 Tujuan Project

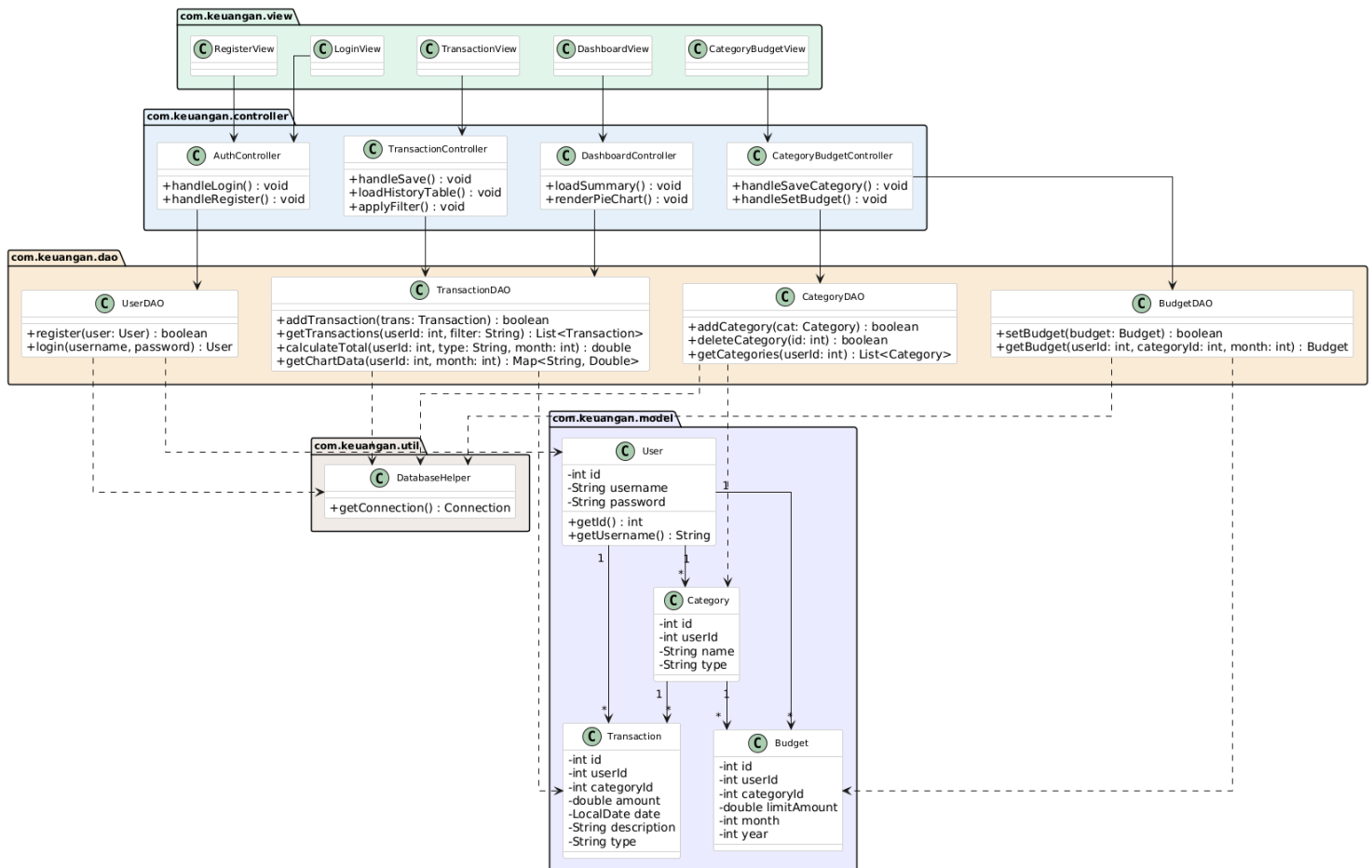
1. Membantu pengguna mencatat setiap transaksi keuangan secara rapi dan terorganisir.
2. Memberikan visualisasi data pengeluaran agar pengguna mudah mengevaluasi kebiasaan belanja.
3. Membantu pengguna mengontrol pengeluaran melalui fitur penetapan anggaran (budgeting).

2. Analisis Kebutuhan

2.1 Fitur Utama Aplikasi

Manajemen Autentikasi	Fitur Login dan Register untuk mengamankan data keuangan pribadi pengguna.
Dashboard Ringkasan	Menampilkan total saldo, total pemasukan, dan total pengeluaran bulan berjalan secara real-time
Pencatatan Pemasukan	Input nominal, tanggal, dan sumber pemasukan (misal: gaji, kiriman orang tua).
Pencatatan Pengeluaran	Input nominal, tanggal, dan keterangan pengeluaran (misal: makan, transportasi).
Kategori Kustom	Pengguna bisa menambah atau menghapus kategori transaksi (misal: "Hobi", "Edukasi").
Set Budget Bulanan	Fitur untuk menetapkan batas maksimal pengeluaran per bulan atau per kategori.
Riwayat Transaksi	Menampilkan daftar semua transaksi yang pernah dilakukan dengan fitur filter berdasarkan tanggal atau kategori.
Laporan Visual (Chart)	Menampilkan grafik pie chart yang menunjukkan persentase pengeluaran berdasarkan kategori.

2.2 Class Diagram dan Relasinya



1. Package View (com.keuangan.view)

Lapisan ini merepresentasikan antarmuka pengguna (UI) yang biasanya dibangun menggunakan file .fxml pada JavaFX. Kelas-kelas di sini tidak memiliki logika pemrosesan data, melainkan murni untuk interaksi visual dengan pengguna.

- LoginView**: Berfungsi menampilkan halaman form masuk (input *username* dan *password*) serta tombol aksi untuk *login*.
- RegisterView**: Berfungsi menampilkan halaman form pendaftaran akun baru bagi pengguna yang belum memiliki akun.
- DashboardView**: Berfungsi menampilkan halaman utama beranda aplikasi yang memuat kartu ringkasan keuangan dan grafik pengeluaran (*Pie Chart*).
- TransactionView**: Berfungsi menampilkan form untuk mencatat pemasukan atau pengeluaran baru, serta menampilkan tabel riwayat transaksi beserta alat filternya.

- e. *CategoryBudgetView*: Berfungsi menampilkan halaman pengaturan di mana pengguna dapat membuat kategori transaksi baru dan menetapkan batas anggaran (*budget*) bulanan.
2. Package Controller (com.keuangan.controller)
- Lapisan ini adalah "otak" dari antarmuka. Bertugas menangkap interaksi pengguna dari *View* (seperti klik tombol), memproses logika dasar, dan memerintahkan *DAO* untuk melakukan operasi *database*.
- a. *AuthController*: Mengendalikan layar autentikasi. Memiliki fungsi *handleLogin()* untuk memvalidasi input masuk pengguna dan *handleRegister()* untuk memproses pendaftaran akun dengan memanggil *UserDAO*.
 - b. *DashboardController*: Mengendalikan layar beranda. Memiliki fungsi *loadSummary()* untuk memanggil dan menampilkan data total uang, serta *renderPieChart()* untuk menggambar grafik visual ke layar dengan data dari *TransactionDAO*.
 - c. *TransactionController*: Mengendalikan operasi pencatatan kas. Memiliki fungsi *handleSave()* untuk menyimpan input transaksi baru, *loadHistoryTable()* untuk menampilkan data ke dalam *TableView*, dan *applyFilter()* untuk menyaring riwayat berdasarkan kategori/tanggal.
 - d. *CategoryBudgetController*: Mengelola konfigurasi kategori dan anggaran. Memiliki fungsi *handleSaveCategory()* untuk menyimpan kategori baru dan *handleSetBudget()* untuk menetapkan batasan limit pengeluaran bulanan.
3. Package DAO (com.keuangan.dao)
- Lapisan *Data Access Object* (DAO) bertugas secara khusus untuk mengeksekusi semua perintah SQL (CRUD: *Create, Read, Update, Delete*) ke *database MySQL*.
- a. *UserDAO*: Menangani kueri tabel pengguna. Fungsi *register()* untuk menyisipkan data akun baru, dan *login()* untuk memeriksa kecocokan *username* dan *password* di *database*.
 - b. *CategoryDAO*: Menangani kueri tabel kategori. Fungsi *addCategory()* untuk membuat kategori kustom, *deleteCategory()* untuk menghapusnya, dan *getCategories()* untuk mengambil daftar kategori milik pengguna tertentu.
 - c. *TransactionDAO*: Pekerja data paling kompleks. Fungsi *addTransaction()* untuk menyimpan transaksi, *getTransactions()* untuk mengambil daftar

riwayat, `calculateTotal()` untuk menghitung total uang (masuk/keluar), dan `getChartData()` untuk merangkum data grafik persentase.

- d. *BudgetDAO*: Menangani kueri tabel anggaran. Fungsi `setBudget()` untuk menyimpan batasan anggaran pengeluaran, dan `getBudget()` untuk menarik data target anggaran bulan berjalan.

4. Package Model (`com.keuangan.model`)

Lapisan ini berisi *Plain Old Java Object* (POJO) yang bertindak sebagai "wadah" atau representasi dari tabel *database* di dalam memori Java. Hanya berisi atribut penyimpanan data dan metode *Getter/Setter*.

- a. *User*: Menyimpan data kredensial pengguna (`id`, `username`, `password`).
- b. *Category*: Menyimpan atribut pengelompokan transaksi (`id`, `userId` pemilik, nama kategori, dan tipe).
- c. *Transaction*: Menyimpan detail pencatatan kas (besaran nominal/amount, `date`, `description`, tipe, dan relasi `categoryId`).
- d. *Budget*: Menyimpan aturan batasan pengeluaran (`limitAmount`, `month`, `year`, dan relasi `categoryId`).

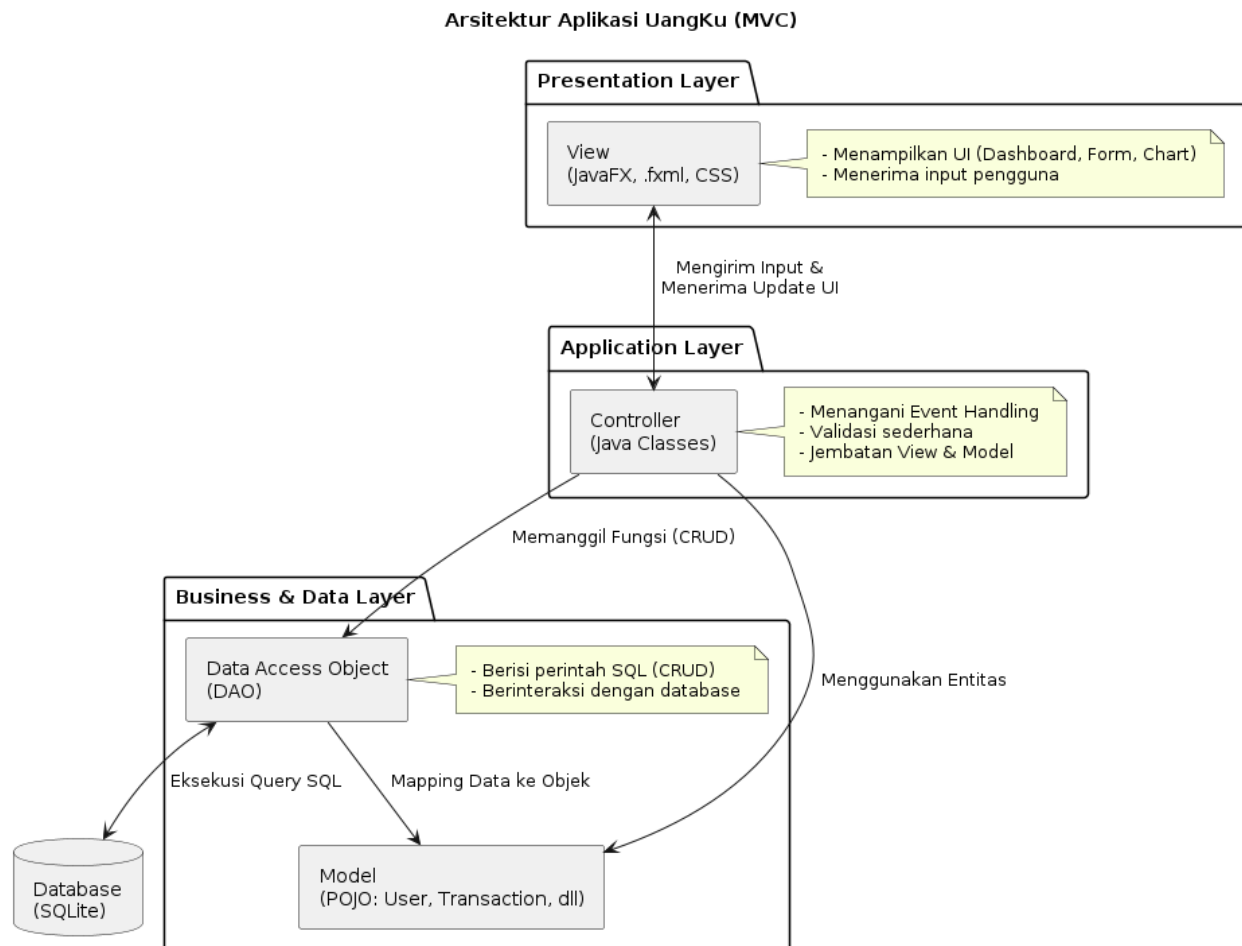
5. Package Util (`com.keuangan.util`)

Lapisan ini berisi kelas-kelas alat bantu (*utility*) yang dapat digunakan oleh seluruh bagian program untuk mencegah penulisan kode yang berulang-ulang.

- a. *DatabaseHelper*: Berfungsi sebagai penyedia koneksi ke server MySQL. Memiliki fungsi `getConnection()` yang akan dipanggil oleh setiap kelas DAO (seperti *UserDAO* atau *TransactionDAO*) setiap kali mereka ingin mengeksekusi *query* SQL.

3. Rancangan Sistem

3.1 Arsitektur Aplikasi



1. Komponen Arsitektur

1. View (Tampilan):

- Teknologi: JavaFX (file .fxml) dan CSS.
- Fungsi: Bagian ini berisi desain antarmuka pengguna (UI) seperti halaman Dashboard, Form Transaksi, dan Chart Laporan. View hanya bertugas menampilkan data kepada pengguna dan mengirimkan input (klik tombol, teks input) ke Controller.

2. Controller (Logika Antarmuka):

- Teknologi: Java Controller Classes.
- Fungsi: Menghubungkan View dengan Model. Controller akan menangani *event handling* (misal: saat tombol "Tambah Pengeluaran" diklik). Controller mengambil data dari input user, memvalidasinya secara

sederhana, lalu memanggil fungsi di lapisan Model untuk diproses lebih lanjut.

3. Model (Logika Bisnis & Data):

- a. Entitas (POJO): Kelas Java seperti User.java, Transaction.java, Category.java, dan Budget.java.
- b. Data Access Object (DAO): Kelas yang berisi perintah SQL untuk berinteraksi dengan SQLite. DAO berfungsi untuk mengambil (Read), menyimpan (Create), memperbarui (Update), dan menghapus (Delete) data dari database.

2. Alur Kerja Aplikasi (Data Flow)

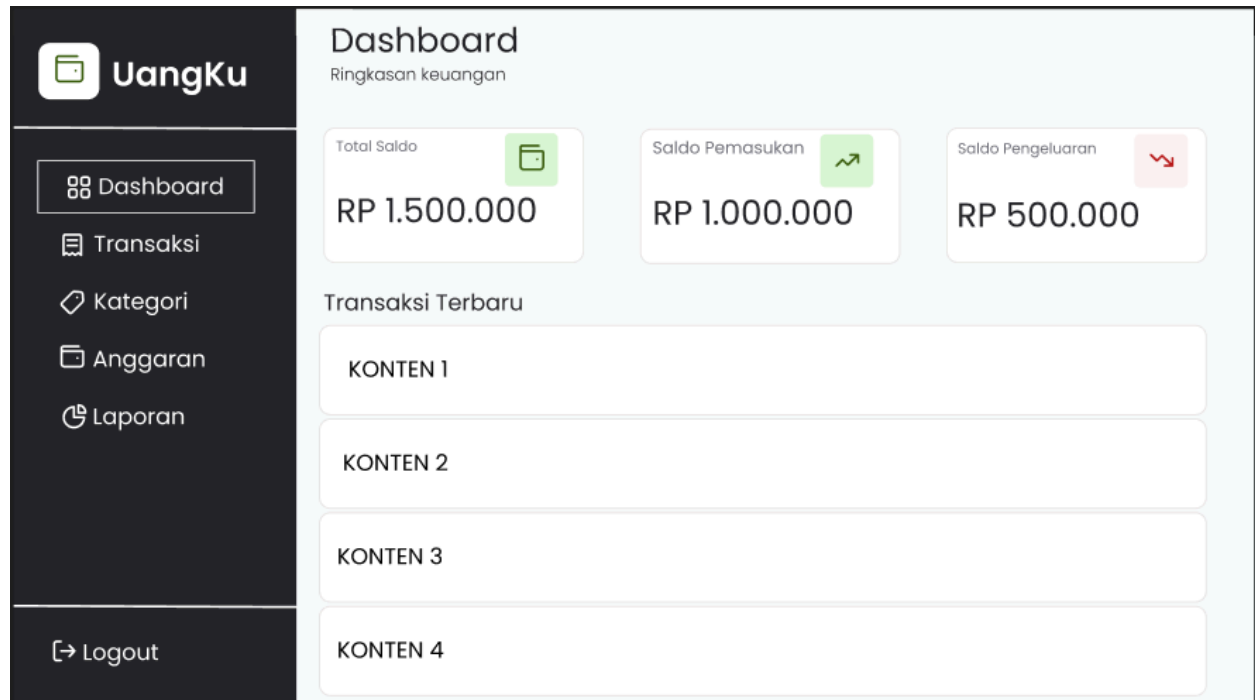
Untuk memberikan gambaran jelas, berikut adalah alur kerja saat pengguna mencatat transaksi baru:

1. Input (View): Pengguna mengisi nominal dan kategori di form pada aplikasi.
2. Handling (Controller): Controller menangkap data tersebut saat tombol simpan diklik.
3. Process (Model/DAO): Controller mengirim data ke kelas TransactionDAO. Kelas ini akan menjalankan perintah SQL INSERT INTO transactions... ke database SQLite.
4. Update (View): Setelah database berhasil diperbarui, Controller memerintahkan View untuk memperbarui tampilan (misalnya, total saldo di Dashboard langsung berubah secara otomatis).

3. Keuntungan Arsitektur Ini untuk Proyek "UangKu"

1. Modularitas: Jika Anda ingin mengubah tampilan (misal mengganti warna atau layout di FXML), Anda tidak perlu menyentuh logika database di DAO.
2. Keamanan: Logika enkripsi password (BCrypt) dapat diletakkan di lapisan Model/Service, terpisah dari kode UI.
3. Kemudahan Debugging: Jika terjadi kesalahan saat menyimpan data, Anda cukup memeriksa bagian DAO tanpa harus bingung mencari di file UI.

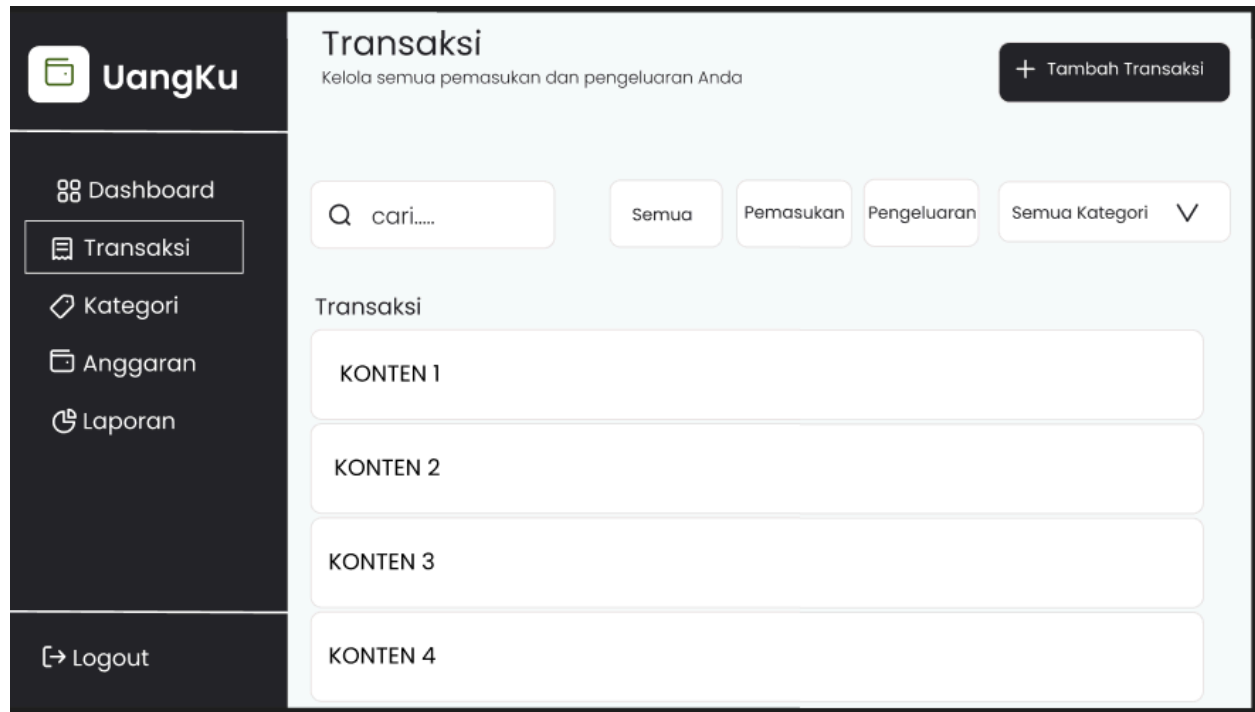
3.2 Wireframe/rancangan UI



Halaman Dashboard

Dashboard berfungsi sebagai pusat informasi utama. Di bagian atas, terdapat tiga kartu ringkasan yang menampilkan:

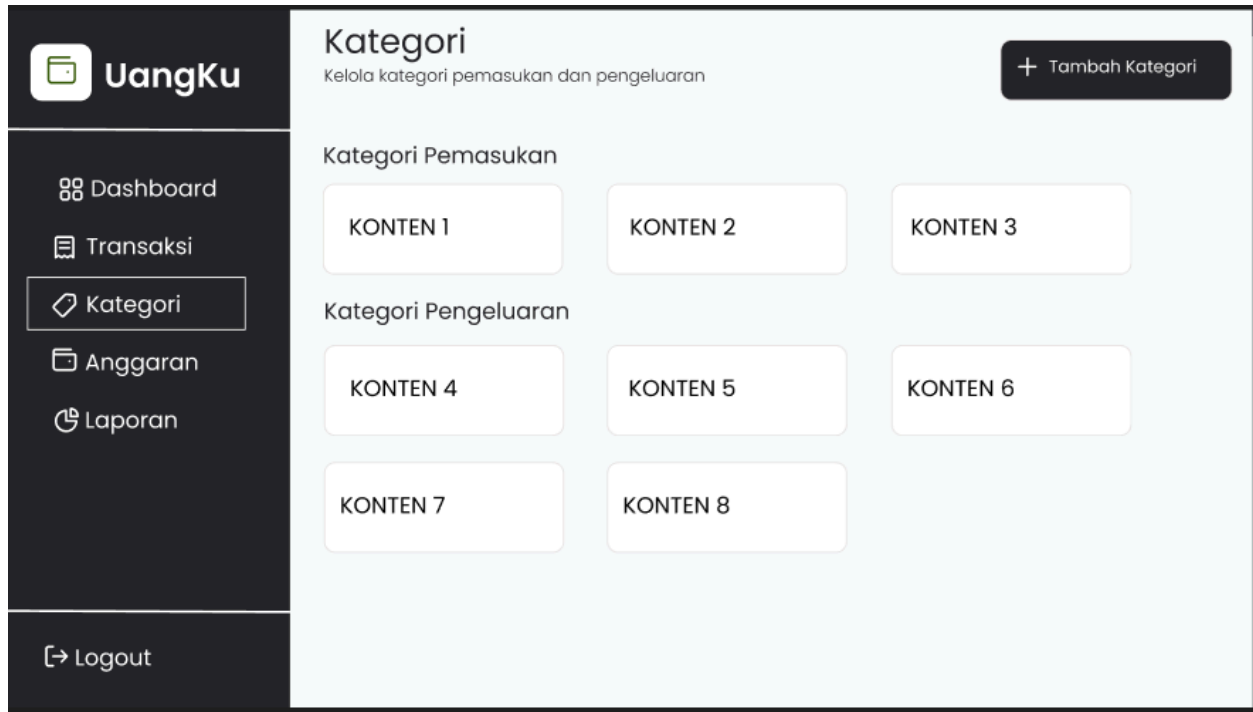
1. Total Saldo: Sisa uang yang dimiliki secara keseluruhan.
2. Saldo Pemasukan: Total uang masuk pada bulan berjalan.
3. Saldo Pengeluaran: Total uang yang telah dibelanjakan.
4. Di bagian bawah kartu, terdapat daftar Transaksi Terbaru untuk memberikan pandangan cepat mengenai aktivitas keuangan terakhir.



Halaman Transaksi

Halaman ini difokuskan pada manajemen data transaksi. Fitur utamanya meliputi:

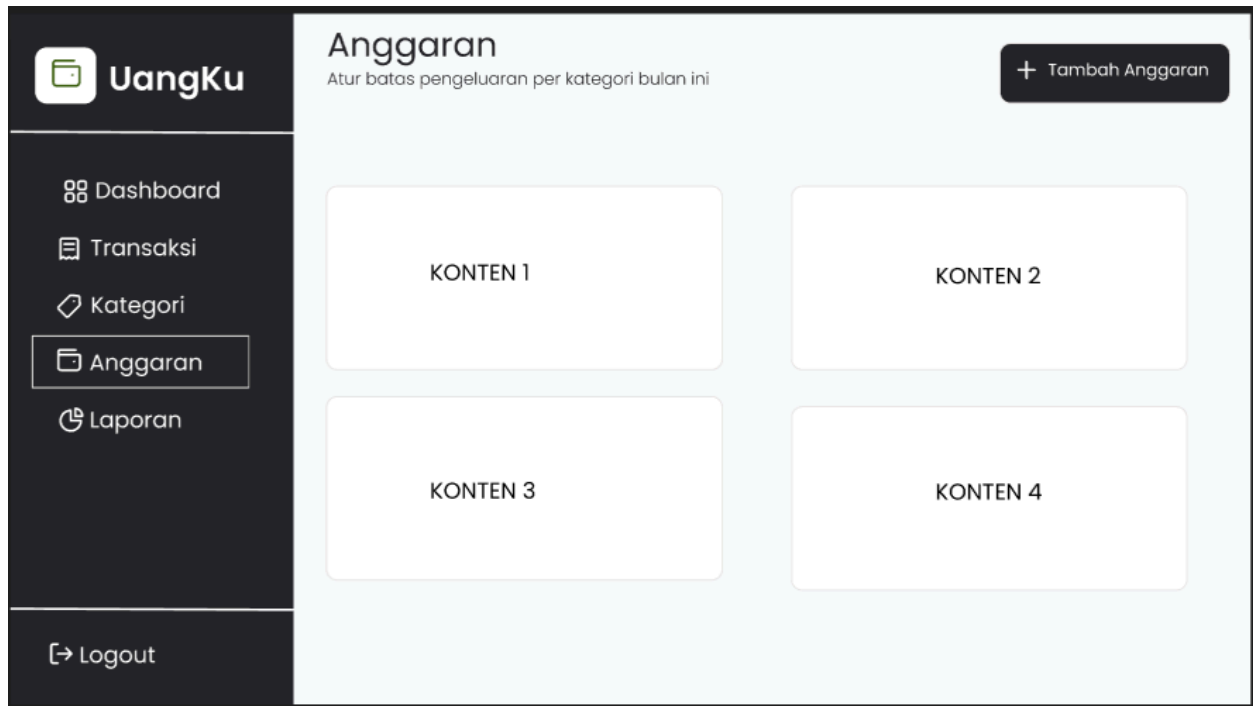
1. Pencarian & Filter: Pengguna dapat mencari transaksi tertentu dan menyaringnya berdasarkan jenis (Pemasukan/Pengeluaran) atau kategori.
2. Tambah Transaksi: Tombol mencolok di pojok kanan atas untuk input data baru.
3. Daftar Riwayat: Menampilkan seluruh catatan transaksi secara kronologis dan rapi.



Halaman Kategori

Halaman ini memungkinkan kustomisasi akun pengguna:

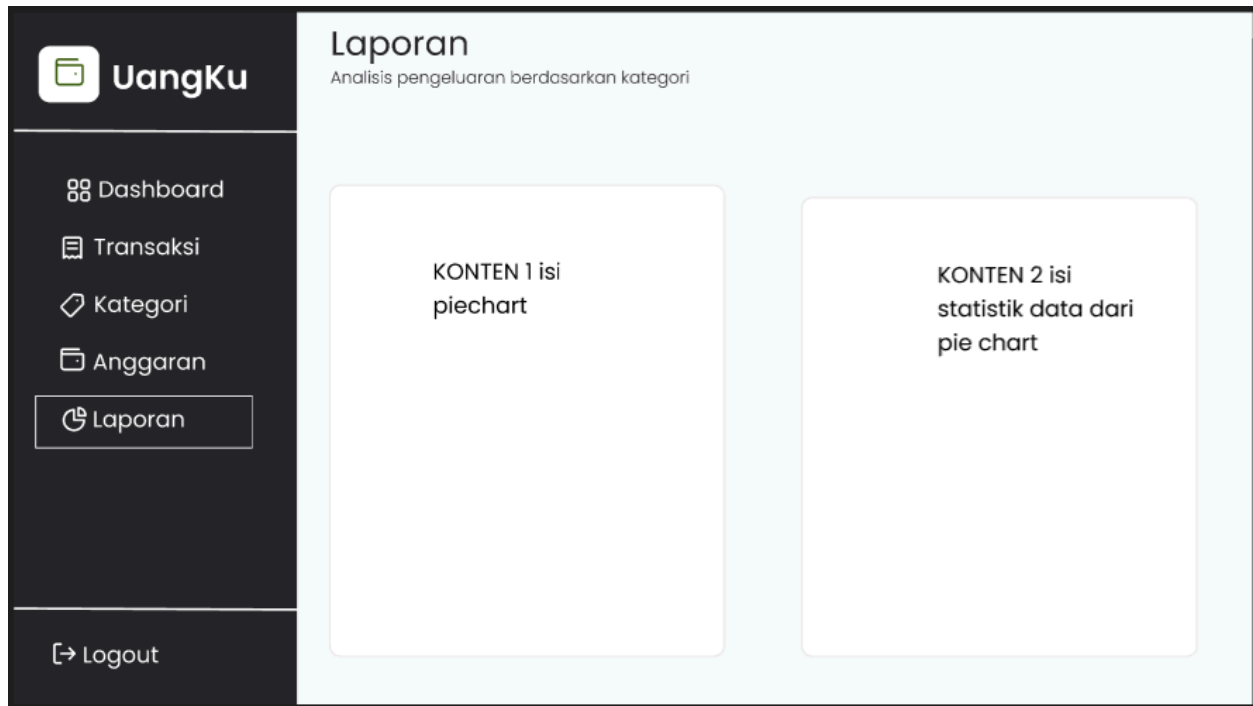
1. Terbagi menjadi dua bagian: Kategori Pemasukan dan Kategori Pengeluaran.
2. Pengguna dapat menambah atau melihat kategori yang sudah ada dalam bentuk kartu (grid) agar lebih mudah dibaca secara visual.



Halaman Anggaran (Budgeting)

Fitur ini membantu pengguna mengontrol pengeluaran dengan menetapkan batas maksimal.

1. Setiap kartu anggaran mewakili satu kategori tertentu.
2. Pengguna dapat memantau apakah pengeluaran mereka masih dalam batas aman atau sudah mendekati limit yang ditentukan.



Halaman Laporan (Visualisasi)

Untuk memudahkan evaluasi keuangan, halaman ini menyajikan data dalam bentuk visual:

1. Pie Chart: Menampilkan persentase pengeluaran berdasarkan kategori untuk melihat ke mana uang paling banyak dihabiskan.
2. Statistik Data: Penjelasan tekstual dari grafik untuk membantu pengguna mengambil keputusan finansial yang lebih baik.

 **UangKu**

Username

Password

Masuk

Belum punya akun? [Daftar](#)

Kelola Keuangan dengan Mudah

Pantau pemasukan, pengeluaran, dan anggaran Anda dalam satu tempat. Simple, elegant, dan efisien.

- Tracking transaksi real-time
- Budget planning per kategori
- Laporan visual yang informatif

Halaman Login: Memiliki antarmuka yang bersih dengan dua kolom input utama (Username dan Password). Terdapat pesan motivasi di sisi kanan untuk menyambut pengguna.

 **UangKu**

Username

Password

Email

Daftar

Sudah punya akun? [Masuk](#)

Kelola Keuangan dengan Mudah

Pantau pemasukan, pengeluaran, dan anggaran Anda dalam satu tempat. Simple, elegant, dan efisien.

- Tracking transaksi real-time
- Budget planning per kategori
- Laporan visual yang informatif

Halaman Register: Digunakan bagi pengguna baru untuk mendaftarkan akun dengan mengisi Username, Password, dan Email. Desain dibuat konsisten dengan halaman login untuk menjaga keselarasan visual.

3.3 Rancangan Struktur Data / Basis Data

Aplikasi UangKu menggunakan SQLite sebagai sistem manajemen basis data, Data akan disimpan dalam satu file database lokal yang dikelola menggunakan library JDBC (Java Database Connectivity).

Berikut adalah detail struktur tabel yang digunakan:

1. Tabel users (Data Pengguna) Tabel ini menyimpan informasi akun untuk sistem autentikasi pengguna.

1. id_user (Integer): Berperan sebagai Primary Key dengan fitur Auto Increment.
2. username (Text): Nama unik pengguna untuk keperluan login.
3. password (Text): Kata sandi pengguna untuk keamanan data.
4. email (Text): Alamat email pengguna.

2. Tabel categories (Kategori Transaksi) Tabel ini menyimpan daftar kategori agar transaksi lebih terorganisir.

1. id_category (Integer): Berperan sebagai Primary Key dengan fitur Auto Increment.
2. id_user (Integer): Berperan sebagai Foreign Key yang merujuk pada tabel users.
3. name (Text): Nama kategori (Contoh: Makanan, Transportasi, Gaji, Bonus).
4. type (Text): Menentukan jenis kategori, apakah "Pemasukan" atau "Pengeluaran".

3. Tabel transactions (Catatan Transaksi) Tabel utama yang mencatat seluruh arus kas masuk dan keluar.

1. id_transaction (Integer): Berperan sebagai Primary Key dengan fitur Auto Increment.
2. id_user (Integer): Berperan sebagai Foreign Key yang merujuk pada tabel users.
3. id_category (Integer): Berperan sebagai Foreign Key yang merujuk pada tabel categories.
4. amount (Real/Double): Nominal uang dalam transaksi tersebut.
5. date (Text): Tanggal transaksi dilakukan (Format: YYYY-MM-DD).
6. description (Text): Keterangan atau catatan tambahan dari pengguna.

4. Tabel budgets (Anggaran) Tabel ini digunakan untuk fitur pembatasan pengeluaran bulanan.

1. **id_budget** (Integer): Berperan sebagai Primary Key dengan fitur Auto Increment.
2. **id_user** (Integer): Berperan sebagai Foreign Key yang merujuk pada tabel users.
3. **id_category** (Integer): Berperan sebagai Foreign Key yang merujuk pada tabel categories.
4. **limit_amount** (Real/Double): Batas maksimal saldo pengeluaran yang ditetapkan.
5. **month_year** (Text): Periode bulan dan tahun berlakunya anggaran.

3.3 Teknologi yang Digunakan

A. Frontend (User Interface):

1. **JavaFX** - Framework untuk membangun antarmuka grafis
2. **FXML** - File XML untuk mendefinisikan layout UI
3. **CSS** - Styling untuk mempercantik tampilan

B. Backend (Business Logic):

1. **Java 11+** - Bahasa pemrograman utama
2. **Java Collections** - Untuk manipulasi data (ArrayList, HashMap, dll)

C. Database:

1. **SQLite** - Database file-based untuk penyimpanan lokal
2. **JDBC (Java Database Connectivity)** - API untuk koneksi Java ke database

D. Libraries yang Digunakan:

1. **sqlite-jdbc** (org.xerial:sqlite-jdbc:3.42.0.0)
 - o Driver JDBC untuk koneksi ke SQLite database
2. **JFoenix** atau **ControlsFX**
 - o Library untuk komponen UI modern (material design)
 - o Membuat tombol, card, dialog yang lebih menarik
3. **JFreeChart** atau **JavaFX Charts API**
 - o Untuk membuat pie chart dan grafik visualisasi
4. **BCrypt** (org.mindrot:jbcrypt:0.4) atau **Jasypt**
 - o Untuk enkripsi password agar aman

E. Build Tools:

1. **Maven** atau **Gradle** - Untuk dependency management dan build automation

4. Pembagian Tugas (Untuk Kelompok)

Nama Anggota	NIM	Tugas & Tanggung Jawab
Waraney Maikel Nathaniel Mambu	71241164	Backend Controller
Delvin Laurens	71241097	Frontend / UI Designer
Valentino Kevin Yulianto	71241126	Database
Jeremy Zadrimman Kause	71241163	Backend Integration

5.Link Figma

<https://www.figma.com/design/4esEJqdFFhIXNZsQv1GSct/Untitled?node-id=0-1&t=VenqfzA1wSZwysM0-1>

6.Link Docs

<https://docs.google.com/document/d/1Y-zJYIPx4kNugqctRs2FwwAsN2izl1do/edit>